

Advanced master theorem for divide and conquer recurrences

Last Updated : 11 Jul, 2025



The Master Theorem is a tool used to solve recurrence relations that arise in the analysis of divide-and-conquer algorithms. The Master Theorem provides a systematic way of solving recurrence relations of the form:

$$T(n) = aT(n/b) + f(n)$$

1. where a , b , and $f(n)$ are positive functions and n is the size of the problem. The Master Theorem provides conditions for the solution of the recurrence to be in the form of $O(n^k)$ for some constant k , and it gives a formula for determining the value of k .
2. The **advanced version** of the Master Theorem provides a more general form of the theorem that can handle recurrence relations that are more complex than the basic form. The advanced version of the Master Theorem can handle recurrences with multiple terms and more complex functions.
3. It is important to note that the Master Theorem is **not applicable to all recurrence relations**, and it may not always provide an exact solution to a given recurrence. However, it is a useful tool for analyzing the time complexity of divide-and-conquer algorithms and provides a good starting point for solving more complex recurrences.

Master Theorem is **used to determine running time of algorithms** (divide and conquer algorithms) in terms of asymptotic notations.

Consider a problem that is solved using recursion.

```
function f(input  $x$  size  $n$ )  
  if( $n < k$ )  
    solve  $x$  directly and return  
  else  
    divide  $x$  into  $a$  subproblems of size  $n/b$  call  
    if recursively to solve each subproblem  
    Combine the results of all sub-problems
```

The above algorithm divides the problem into **a** subproblems, each of size n/b and solve them recursively to compute the problem and the extra work done for problem is given by $f(n)$, i.e., the time to create the subproblems and combine their results in the above procedure.

So, according to master theorem the runtime of the above algorithm can be expressed as:

$$T(n) = aT(n/b) + f(n)$$

where n = size of the problem

a = number of subproblems in the recursion and $a \geq 1$

n/b = size of each subproblem

$f(n)$ = cost of work done outside the recursive calls like dividing into subproblems and cost of combining them to get the solution.

Not all recurrence relations can be solved with the use of the master theorem i.e. if

- *$T(n)$ is not monotone, ex: $T(n) = \sin n$*
- *$f(n)$ is not a polynomial, ex: $T(n) = 2T(n/2) + 2^n$*

This theorem is an advance version of master theorem that can be used to determine running time of divide and conquer algorithms if the recurrence is of the following form :-

$$T(n) = aT(n/b) + \theta(n^k \log^p n)$$

where n = size of the problem

a = number of subproblems in the recursion and $a \geq 1$

n/b = size of each subproblem

$b > 1$, $k \geq 0$ and p is a real number.

Then,

1. if $a > b^k$, then $T(n) = \theta(n^{\log_b a})$
2. if $a = b^k$, then
 - (a) if $p > -1$, then $T(n) = \theta(n^{\log_b a} \log^{p+1} n)$
 - (b) if $p = -1$, then $T(n) = \theta(n^{\log_b a} \log \log n)$
 - (c) if $p < -1$, then $T(n) = \theta(n^{\log_b a})$
3. if $a < b^k$, then
 - (a) if $p \geq 0$, then $T(n) = \theta(n^k \log^p n)$
 - (b) if $p < 0$, then $T(n) = \theta(n^k)$

Time Complexity Analysis -

- **Example-1: Binary Search** - $T(n) = T(n/2) + O(1)$

$a = 1$, $b = 2$, $k = 0$ and $p = 0$

$b^k = 1$. So, $a = b^k$ and $p > -1$ [Case 2.(a)]

$$T(n) = \theta(n^{\log_b a} \log^{p+1} n)$$

$$T(n) = \theta(\log n)$$

- **Example-2: Merge Sort** - $T(n) = 2T(n/2) + O(n)$

$a = 2$, $b = 2$, $k = 1$, $p = 0$

$b^k = 2$. So, $a = b^k$ and $p > -1$ [Case 2.(a)]

$$T(n) = \theta(n^{\log_b a} \log^{p+1} n)$$

$$T(n) = \theta(n \log n)$$

- **Example-3:** $T(n) = 3T(n/2) + n^2$

$a = 3$, $b = 2$, $k = 2$, $p = 0$

$b^k = 4$. So, $a < b^k$ and $p = 0$ [Case 3.(a)]

$$T(n) = \theta(n^k \log^p n)$$

$$T(n) = \theta(n^2)$$

- **Example-4:** $T(n) = 3T(n/2) + \log^2 n$

$a = 3$, $b = 2$, $k = 0$, $p = 2$

$b^k = 1$. So, $a > b^k$ [Case 1]

$$T(n) = \theta(n^{\log_b a})$$

$$T(n) = \theta(n^{\log_2 3})$$

- **Example-5:** $T(n) = 2T(n/2) + n\log^2 n$

$$a = 2, b = 2, k = 1, p = 2$$

$$b^k = 2. \text{ So, } a = b^k \text{ [Case 2.(a)]}$$

$$T(n) = \theta(n^{\log_b a} \log^{p+1} n)$$

$$T(n) = \theta(n^{\log_2 2} \log^3 n)$$

$$T(n) = \theta(n \log^3 n)$$

- **Example-6:** $T(n) = 2^n T(n/2) + n^n$

This recurrence can't be solved using above method since function is not of form

$$T(n) = aT(n/b) + \theta(n^k \log^p n)$$

GATE Practice questions -

- [GATE-CS-2017 \(Set 2\) | Question 56](#)
- [GATE IT 2008 | Question 42](#)
- [GATE CS 2009 | Question 35](#)

Here are some important points to keep in mind regarding the Master Theorem:

1. Divide-and-conquer recurrences: The Master Theorem is specifically designed to solve recurrence relations that arise in the analysis of divide-and-conquer algorithms.
2. Form of the recurrence: The Master Theorem applies to recurrence relations of the form $T(n) = aT(n/b) + f(n)$, where a , b , and $f(n)$ are positive functions and n is the size of the problem.
3. Time complexity: The Master Theorem provides conditions for the solution of the recurrence to be in the form of $O(n^k)$ for some constant k , and it gives a formula for determining the value of k .
4. Advanced version: The advanced version of the Master Theorem provides a more general form of the theorem that can handle recurrence relations that are more complex than the basic form.

5. Limitations: The Master Theorem is not applicable to all recurrence relations, and it may not always provide an exact solution to a given recurrence.
6. Useful tool: Despite its limitations, the Master Theorem is a useful tool for analyzing the time complexity of divide-and-conquer algorithms and provides a good starting point for solving more complex recurrences.
7. Supplemented with other techniques: In some cases, the Master Theorem may need to be supplemented with other techniques, such as the substitution method or the iteration method, to completely solve a given recurrence relation.

[Comment](#)[More info](#) ▼[Advertise with us](#)

Explore

[DSA Fundamentals](#)[Data Structures](#)[Algorithms](#)[Advanced](#)[Interview Preparation](#)[Practice Problem](#)

Do Not Sell or Share My Personal Information